



Gabriel Alejandro Molina Quintana
Software Engineer

PDF generated at: 5 Apr 2026 19:27:41 UTC
[View this report on HackerRank](#)

Score

88.6% • 155.0 / 175
scored in Atenea Mercantil Software Engineer in 82 min 35 sec on 5 Apr 2026 14:02:50 AST

Candidate Information

Email	gabriel@ateneamercantil.com
Test	Atenea Mercantil Software Engineer
Candidate Packet	View
Taken on	5 Apr 2026 14:02:50 AST
Time taken	82 min 35 sec/ 90 min
Work Experience	2 years
Invited by	Talento
Invited on	31 Mar 2026 13:00:53 AST

Skill Distribution

No.	Skill	Score
1	Problem Solving Basic	100%

2	SQL Basic	100%	
3	REST API Intermediate	73%	

Tags Distribution

Easy	100%	Sets	100%
Queues	100%	Sorting	84%
Greedy Algorithms	100%	SQL	100%
Interviewer Guidelines	84%	Simple Joins	100%
Database	100%	REST API	73%
Back-End Development	73%	Medium	73%
JSON	73%	HTTP	73%

Questions

Problem Solving (Basic) • 50 / 50

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	Query Optimization Coding	Problem Solving (Basic)	50/50	-	29 min 1 sec	

SQL (Basic) • 50 / 50

No.	Question	Skill	Score	Code Quality	Time Taken	Status
2	Youngest Employees DbRank	SQL (Basic)	50/50	-	24 min 10 sec	

REST API (Intermediate) • 55 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
3	REST API: Movie Titles Coding	REST API (Intermediate)	55/75	-	29 min 8 sec	

1. Query Optimization

 Correct

Coding Easy Sets Queues Sorting Greedy Algorithms

Question description

You are given an array *queryTimes* of size *n*, where *queryTimes[i]* is the execution time of the *ith* query. You may apply the following tuning operation any number of times:

- Choose an even execution time value *t*.
- For every query whose execution time is exactly *t*, update its time to *t* / 2.

Your task is to find the minimum number of tuning operations needed so that all query execution times become odd.

Example

n = 4

queryTimes = [2, 6, 8, 16]

An optimal series of operations

<i>Current queryTimes</i>	<i>Tuning Operation (t)</i>	<i>Queries with execution time = t</i>	<i>Modified queryTimes</i>
[2, 6, 8, 16]	16	[2, 6, 8, 16]	[2, 6, 8, 8]
[2, 6, 8, 8]	8	[2, 6, 8, 8]	[2, 6, 4, 4]
[2, 6, 4, 4]	6	[2, 6, 4, 4]	[2, 3, 4, 4]
[2, 3, 4, 4]	4	[2, 3, 4, 4]	[2, 3, 2, 2]
[2, 3, 2, 2]	2	[2, 3, 2, 2]	[1, 3, 1, 1]

After Step 5, all the execution times are odd. Return 5, the minimum number of tuning operations required.

Constraints

- $1 \leq n \leq 2 * 10^5$
- $1 \leq queryTimes[i] \leq 10^9$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, n , the number of elements in *queryTimes*.
Each of the next n lines contains an integer *queryTimes*[i].

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
STDIN      FUNCTION
-----
2  → queryTimes[] size n = 2
3  → queryTimes = [3, 24]
24
```

Sample Output

```
3
```

Explanation

An optimal series of operations

<i>Current queryTimes</i>	<i>Tuning Operation (t)</i>	<i>Queries with execution time = t</i>	<i>Modified queryTimes</i>
[3, 24]	24	[3, 24]	[3, 12]
[3, 12]	12	[3, 12]	[3, 6]
[3, 6]	6	[3, 6]	[3, 3]

After Step 3, all the execution times are odd.

▼ SAMPLE CASE 1

Sample Input For Custom Testing

```
STDIN      FUNCTION
-----
3   →  queryTimes[] size n = 3
1   →  queryTimes = [1, 9, 5]
9
5
```

Sample Output

```
0
```

Explanation

Since all the queries already have odd execution times, no tuning operation is needed.

Interviewer guidelines

▼ HINT 1

Skills: Sorting, Greedy

Sort the execution times in descending order and use a data structure that allows you to efficiently access and remove the largest even execution time.

▼ HINT 2

After each operation of halving the largest even execution time, check if the result is still even. If it is, you need to perform another operation on it in the future.

▼ BRUTE FORCE SOLUTION

The brute-force solution would involve repeatedly scanning the entire array to find the largest even execution time, halving it, and then checking if the result is still even. This process would continue until all execution times are odd.

```
def getMinTuningOperations(queryTimes):
    ans = 0
    while any(c % 2 == 0 for c in queryTimes):
        largest_even = max((c for c in queryTimes if c % 2 == 0), default=0)
        queryTimes = [c // 2 if c == largest_even else c for c in queryTimes]
        ans += 1
    return ans
```

▼ OPTIMAL SOLUTION

The optimal solution involves sorting the execution times in descending order and using a set or priority queue to keep track of the even execution times that still need to be halved. In each iteration, we remove the largest even execution time, halve it, and if the result is still even, we add it back to the set or priority queue. We continue this process until there are no more even execution times left.

```
def getMinTuningOperations(queryTimes):
    queryTimes.sort(reverse=True)
    s1 = set()
    ans = 0
    while queryTimes:
        it = queryTimes.pop()
        if it % 2 == 0 and it not in s1:
            ans += 1
            s1.add(it)
            queryTimes.append(it // 2)
    return ans
```

▼ COMPLEXITY ANALYSIS

Time Complexity (Brute Force) - $O(n^2)$

For each operation, we scan the entire array to find the largest even execution time

Space Complexity (Brute Force) - $O(1)$

No additional data structures depend on the size of the input.

Time Complexity (Optimal) - $O(n * \log(n))$

Execution times are sorted and the operations are performed on the set or priority queue

Space Complexity (Optimal) - $O(n)$

Execution times are stored in a set or priority queue.

▼ FOLLOW UP QUESTION

Follow-Up Question 1: What if the operation could be applied to any layer, not just the ones with even execution times, and it would reduce the execution time by one third instead of half?

Solution: The solution would need to be adjusted to account for the new operation rule. Instead of checking for even execution times, we would need to apply the operation to the largest execution time and reduce it by one third. The data structure and the overall approach would remain similar, but the condition for applying the operation and the reduction amount would change.

Follow-Up Question 2: What if we could only apply the operation a maximum of k times, and we want to minimize the sum of execution times after k operations?

Solution: This would require a modification to the greedy approach. Instead of aiming to make all execution times odd, we would prioritize the layers with the highest execution times and apply the operation to them first. We would keep track of the number of operations and stop once we reach k . The goal would be to minimize the sum, so we might need to use a different strategy to decide which layers to choose for the operation.

Candidate's Solution

Language used: JavaScript (Node.js)

```
1  'use strict';
2
3  const fs = require('fs');
4
5  process.stdin.resume();
6  process.stdin.setEncoding('utf-8');
7
8  let inputString = '';
9  let currentLine = 0;
10
11 process.stdin.on('data', function(inputStdin) {
12     inputString += inputStdin;
13 });
14
15 process.stdin.on('end', function() {
```

```
16   inputString = inputString.split('\n');
17
18   main();
19 });
20
21 function readLine() {
22   return inputString[currentLine++];
23 }
24
25
26
27 /*
28  * Complete the 'getMinTuningOperations' function below.
29  *
30  * The function is expected to return an INTEGER.
31  * The function accepts INTEGER_ARRAY queryTimes as parameter.
32  */
33
34 function getMinTuningOperations(queryTimes) {
35
36   const repeat = new Set();
37   let operation = 0;
38
39   for(let num of queryTimes) {
40     while(num % 2 === 0){
41       if(!repeat.has(num)) {
42         repeat.add(num);
43         operation++
44       }
45       num = num / 2
46     }
47   }
48
49   return operation;
50
51 }
52
53 function main() {
54   const ws = fs.createWriteStream(process.env.OUTPUT_PATH);
55
56   const queryTimesCount = parseInt(readLine().trim(), 10);
57
58   let queryTimes = [];
59
60   for (let i = 0; i < queryTimesCount; i++) {
61     const queryTimesItem = parseInt(readLine().trim(), 10);
```

```
62     queryTimes.push(queryTimesItem);
63 }
64
65     const result = getMinTuningOperations(queryTimes);
66
67     ws.write(result + '\n');
68
69     ws.end();
70 }
71
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.0396 sec	38.5 KB
TestCase 1	Easy	Sample	Success	1	0.0351 sec	39 KB
TestCase 2	Easy	Sample	Success	1	0.0371 sec	38.8 KB
TestCase 3	Easy	Hidden	Success	2	0.0427 sec	38.6 KB
TestCase 4	Easy	Hidden	Success	3	0.0347 sec	38.9 KB
TestCase 5	Medium	Hidden	Success	3	0.0375 sec	39.3 KB
TestCase 6	Medium	Hidden	Success	3	0.0355 sec	38.8 KB
TestCase 7	Medium	Hidden	Success	3	0.0325 sec	39.1 KB
TestCase 8	Medium	Hidden	Success	3	0.0569 sec	39.5 KB

TestCase 9	Hard	Hidden	Success	5	0.1514 sec	84.7 KB
TestCase 10	Hard	Hidden	Success	5	0.1031 sec	61.5 KB
TestCase 11	Hard	Hidden	Success	5	0.396 sec	119 KB
TestCase 12	Hard	Hidden	Success	5	0.1499 sec	75 KB
TestCase 13	Hard	Hidden	Success	5	0.7035 sec	191 KB
TestCase 14	Hard	Hidden	Success	5	0.2041 sec	86.8 KB

No comments.

2. Youngest Employees

Correct

DbRank

SQL

Easy

Interviewer Guidelines

Simple Joins

Database

Question description

Write a query to generate a list of all employees who are less than 25 years old.

Sort the results first by *NAME*, then by employee *ID*, both in ascending order. The result should include the *UIN* followed by the *NAME*.

▼ SCHEMA

EMPLOYEE		
Name	Type	Description
ID	Integer	The ID of the employee. This is a primary key.
NAME	String	The name of the employee having [1, 20] characters.
AGE	Integer	The age of the employee.
ADDRESS	String	The address of the employee having [1, 25] characters.
SALARY	Integer	The salary of the employee.

EMPLOYEE_UIN		
Name	Type	Description
ID	Integer	The ID of the employee. This is a primary key.
UIN	String	The unique identification number of the employee.

▼ SAMPLE DATA TABLES

Sample Input

EMPLOYEE				
ID	NAME	AGE	ADDRESS	SALARY
1	Sherrie	23	Paris	74635
2	Paul	30	Sydney	72167
3	Mary	24	Paris	75299
4	Sam	47	Sydney	46681
5	Dave	22	Texas	11843

EMPLOYEE_UIN

ID	UIN
1	57520-0440
2	49638-001
3	63550-194
4	68599-6112
5	63868-453

Sample Output

```
63868-453 Dave
63550-194 Mary
57520-0440 Sherrie
```

Explanation

- *Sherrie* is 23 years old and has UIN 57520-0440. This record is printed.
- *Paul* is 30 years old and has UIN 49638-001. This record is not printed.
- A similar analysis is done on the remaining records.

None of the three names of people less than 25 years old is repeated, so print them in alphabetical order. There is no additional sorting by *ID* in this case.

Interviewer guidelines

▼ SOLUTION

Concepts covered: e.g. JOIN, ORDER BY

Solution:

Join the tables to get UIN. Filter results to age < 25 and sort ascending by name, id.

```
SELECT eu.uin, e.name
FROM employee e
```

```
JOIN employee_uin eu
ON e.id = eu.id
WHERE e.age < 25
ORDER BY e.name, e.id;
```

Note that the secondary sort is on ID, but data reported is UIN.

Candidate's Solution

Language used: **MySQL**

```
1
2 SELECT Employee_uin.uin, Employee.name FROM Employee JOIN Employee_uin ON
   Employee_uin.id = Employee.id WHERE Employee.age < 25 ORDER BY Employee.name
   ASC, Employee.id ASC
3
```

Time taken: 0.01 sec

🚫 No comments.

3. REST API: Movie Titles

✅ Partially correct

Coding

REST API

Back-End Development

Medium

JSON

Interviewer Guidelines

HTTP

Sorting

Question description

Write a function to retrieve and sort movie titles from an API.

Implement the *getMovieTitles* function that accepts a string parameter *substr* and:

1. Queries the URL *https://jsonmock.hackerrank.com/api/moviesdata/search/?Title=<substr>*.
2. Initializes an array *titles* to store the movie titles.
3. Extracts the *Title* field from each record in the *data* array and stores it in the *titles* array.
4. Sorts the *titles* array alphabetically in ascending order and returns it.

Note: For additional pages, append *&page=<pageNumber>*. For example, to request the second page of results where *Title* contains 'and', the URL is *https://jsonmock.hackerrank.com/api/moviesdata/search/?*

Title=and&page=2.

The query response from the website is a JSON response with the following five fields:

- *page* : The current page
- *per_page* : The maximum number of results per page
- *total* : The total number of records in the search result
- *total_pages* : The total number of pages which must be queried to get all the results
- *data* : An array of JSON objects containing movie information

Each *data* array object has the following 3 fields:

- *Title*: the movie title
- *Year*: year of release
- *imdbID*: a unique identifier

Function Description

Complete the *getMovieTitles* function in the editor with the following parameter(s):

string substr: the substring to search for

Returns:

string[]: an array of strings, *titles* sorted in ascending order

Note: Please review the header in the code stub to see available libraries for API requests in the selected language. Required libraries can be imported in order to solve the question. Check our full list of supported libraries at <https://www.hackerrank.com/environment>.

▼ INPUT FORMAT FOR CUSTOM TESTING

The only line contains a string, *substr*.

▼ SAMPLE CASE 0

Sample Input

STDIN	Function
spiderman	→ substr = 'spiderman'

Sample Output

Fighting, Flying and Driving: The Stunts of Spiderman 3
Italian Spiderman
Spiderman
Spiderman 5
Spiderman in Cannes
Superman, Spiderman or Batman

Explanation

For this example, all titles that contain the substring `spiderman` should be displayed. The response for the query `https://jsonmock.hackerrank.com/api/moviesdata/search/?Title=spiderman&page=1` is:

```
{"page":1,"per_page":10,"total":6,"total_pages":1,"data":[{"Title":"Italian Spiderman","Year":2007,"imdbID":"tt2705436"}, {"Title":"Superman, Spiderman or Batman","Year":2011,"imdbID":"tt2084949"}, {"Title":"Spiderman","Year":1990,"imdbID":"tt0100669"}, {"Title":"Fighting, Flying and Driving: The Stunts of Spiderman 3","Year":2007,"imdbID":"tt1132238"}, {"Title":"Spiderman 5","Year":2008,"imdbID":"tt3696826"}, {"Title":"Spiderman in Cannes","Year":2016,"imdbID":"tt5978586"}]}
```

Note: Since there are only 6 titles, there is nothing on page 2.

The values of the `Title` field for each record in the order received are:

Italian Spiderman
Superman, Spiderman or Batman
Spiderman
Fighting, Flying and Driving: The Stunts of Spiderman 3
Spiderman 5
Spiderman in Cannes

Sort the array in ascending order, and return it as the answer.

Interviewer guidelines

▼ SOLUTION

```
def getMovieTitles(substr):  
    page = 0  
    pages = 1  
    titles = []
```

```
while page < pages:
    page += 1
    result = requests.get(f"https://jsonmock.hackerrank.com/api/moviesdata/search/?Title={subst
r}&page={page}")
    obj = result.json()
    if page == 1:
        pages = obj.get('total_pages')
    for data in obj.get('data'):
        titles.append(data['Title'])
return sorted(titles)
```

Candidate's Solution

Language used: JavaScript (Node.js)

```
1  'use strict';
2
3  const fs = require('fs');
4  const https = require('https');
5  const axios = require('axios');
6  const fetch = require('node-fetch');
7
8  process.stdin.resume();
9  process.stdin.setEncoding('utf-8');
10
11 let inputString = '';
12 let currentLine = 0;
13
14 process.stdin.on('data', function(inputStdin) {
15     inputString += inputStdin;
16 });
17
18 process.stdin.on('end', function() {
19     inputString = inputString.split('\n');
20     main();
21 });
22
23 function readLine() {
24     return inputString[currentLine++];
25 }
26 /*
27  * Complete the function below.
28  * Base url: https://jsonmock.hackerrank.com/api/moviesdata/search/?Title=
29  */
```

```
30 async function getMovieTitles(substr) {
31
32     const moviesTitle = [];
33     let page = 1;
34     let totalPages = 1;
35
36     while ( page <= totalPages ) {
37         const url = `https://jsonmock.hackerrank.com/api/moviesdata/search/?
Title=${substr}&${page}`;
38         const response = await fetch(url);
39         const jsonResponse = await response.json();
40
41         totalPages = jsonResponse.total_pages;
42
43         for (let movies of jsonResponse.data) {
44             moviesTitle.push(movies.Title);
45         }
46
47         page++;
48     }
49
50     return moviesTitle.sort();
51
52
53 }
54
55 async function main() {
56     const ws = fs.createWriteStream(process.env.OUTPUT_PATH);
57
58     const _substr = readline().trim();
59     const results = await getMovieTitles(_substr);
60     if (results.length > 0) {
61         for (const result of results) {
62             ws.write(`${result}\n`);
63         }
64     }
65
66     ws.end();
67 }
68
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
----------	------------	------	--------	-------	------------	-------------

TestCase 0	Easy	Sample	Success	1	0.2458 sec	55.1 KB
TestCase 1	Easy	Sample	Success	1	0.2571 sec	54.8 KB
TestCase 2	Easy	Hidden	Success	8	0.2485 sec	53.4 KB
TestCase 3	Easy	Hidden	Success	8	0.3016 sec	55.2 KB
TestCase 4	Easy	Hidden	Success	9	0.2442 sec	55 KB
TestCase 5	Easy	Hidden	Success	9	0.2367 sec	55 KB
TestCase 6	Easy	Hidden	Success	9	0.2464 sec	55.3 KB
TestCase 7	Easy	Hidden	Success	10	0.2439 sec	55.1 KB
Testcase 8	Hard	Hidden	Wrong Answer	0	1.084 sec	55.9 KB
Testcase 9	Hard	Hidden	Wrong Answer	0	0.6998 sec	53.4 KB

No comments.

